OPSFLOW

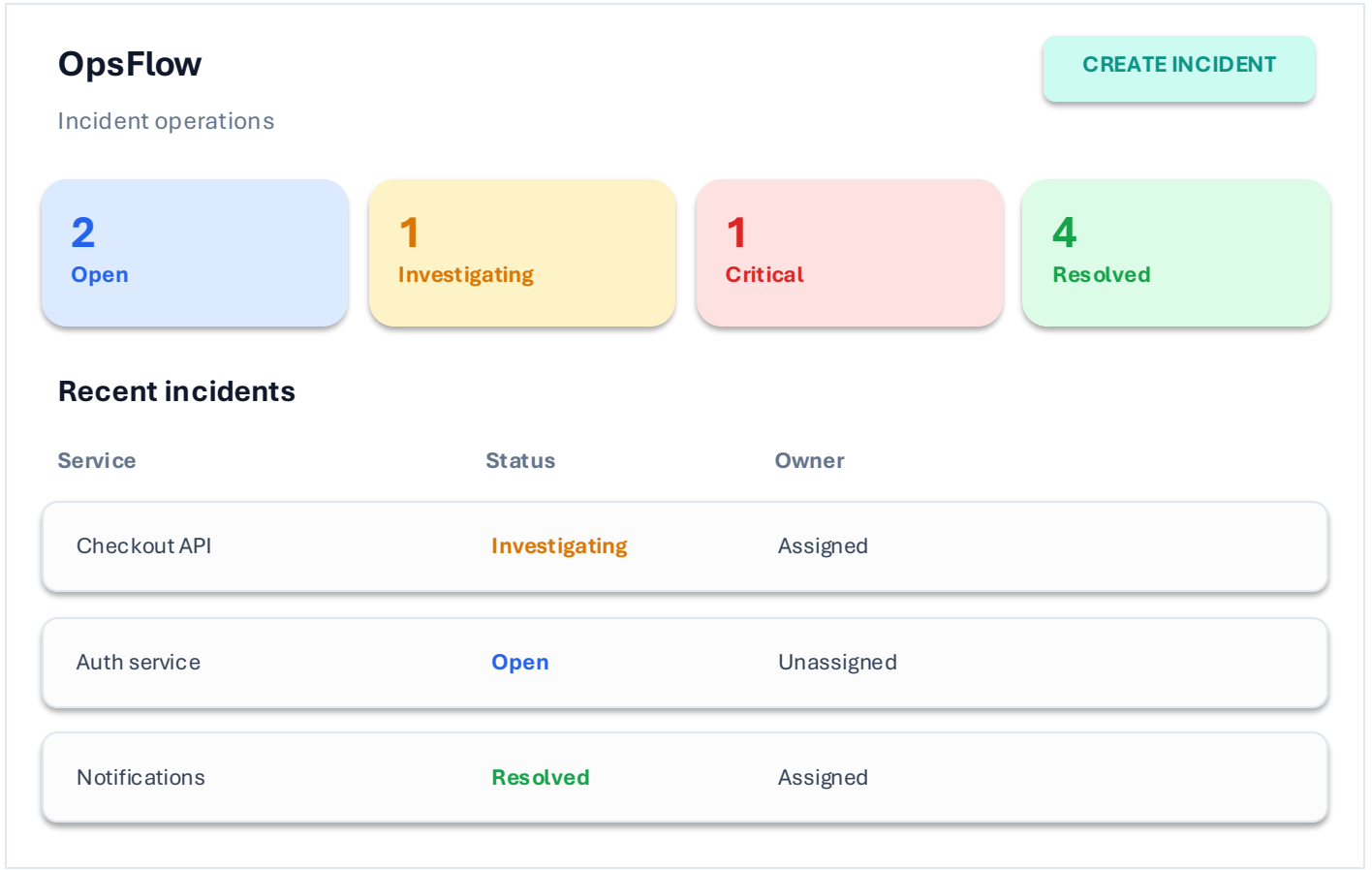
Service incident

operations, made clear and reliable

A simple walkthrough of the application, delivery workflow, cloud validation, GitOps, and monitoring.

Lavanya Soni | 2026

OpsFlow at a glance



Track incidents • coordinate owners • monitor service health

RUNNING APPLICATION

Project overview

A single place to track service problems and the engineering workflow that keeps the platform dependable.

The problem

Operational incidents become expensive when status, ownership, and service health are scattered across tools and conversations.

The solution

OpsFlow brings incident tracking, clear ownership, health checks, and operational visibility into one application.

Why it matters

Faster coordination, safer releases, repeatable deployment, and clearer evidence when something goes wrong.

What a user can do

1. **Create** — Log a service incident
2. **Assign** — Make ownership visible
3. **Investigate** — Track progress and severity
4. **Resolve** — Close with a clear status

Built beyond the UI

- **Quality checks** — 36 automated API tests
- **Security gate** — Serious image vulnerabilities block release
- **Cloud proof** — Validated on AWS EKS, then destroyed
- **Operational insight** — Prometheus + Grafana show health and traffic

Technology stack

Each tool has one clear job in the overall system.

User experience

React + Vite

Browser dashboard for incident work

Application logic

FastAPI + SQLAlchemy

API rules, validation, health and metrics

Data

PostgreSQL + Alembic

Incident storage and controlled schema changes

Containers

Docker + Nginx

Consistent packaging from laptop to cluster

Delivery & security

GitHub Actions + Trivy + GHCR

Tests, image builds, vulnerability gate and registry

Orchestration & GitOps

Kubernetes + Helm + ArgoCD

Keep the desired deployment aligned with Git

Cloud

Terraform + AWS EKS

Repeatable infrastructure and real cloud validation

Observability

Prometheus + Grafana

See availability, traffic and application behavior

build the app → package it → check it → deploy it → keep it aligned → watch it.

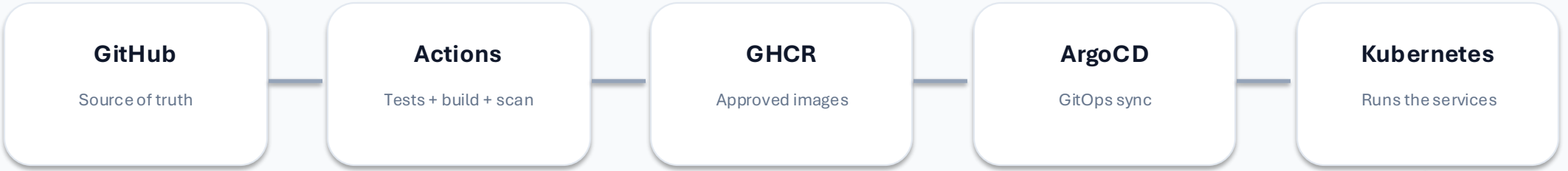
Architecture overview

The application path is simple; the delivery path adds automation, cloud infrastructure and monitoring around it.

APPLICATION PATH



DELIVERY & OPERATIONS



Terraform created the AWS VPC + EKS environment used for cloud validation. After testing, those resources were destroyed to control cost.

Prometheus + Grafana | operational visibility

CI/CD and security flow

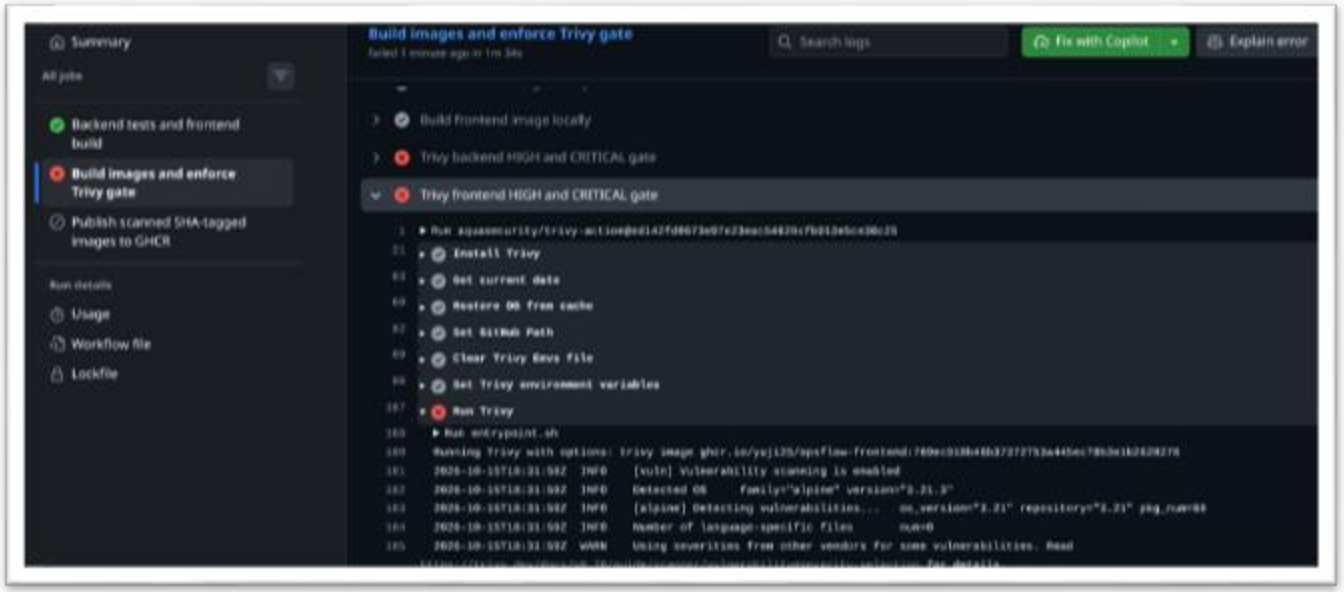
Every change is checked before an image is allowed into the registry.

1 — Test
36 API tests + frontend build

2 — Build
Create backend + frontend images

3 — Scan
Block HIGH / CRITICAL findings

4 — Publish
Push SHA-tagged images to GHCR



Real security behavior: the pipeline stopped when the vulnerability gate failed.

- **Fail closed —** serious findings stop publication.
- **Fix, then rerun —** the dependency/base-image issue was remediated instead of bypassing the gate.
- **Traceable releases —** published images use the full Git commit SHA, not “latest”.

WHY THIS MATTERS: bad builds do not quietly move forward

Infrastructure as code

Cloud infrastructure was created from code, validated on AWS, and then removed after testing.

Repeatable setup

Terraform defines the network and EKS cluster instead of relying on manual console clicks.

Real AWS validation

A VPC, two public subnets, EKS and a managed worker node were provisioned in ap-south-1.

Cost-aware cleanup

The AWS environment was intentionally destroyed after validation to avoid ongoing charges.

Validation record

2
public subnets

1
EKS worker

1.36
Kubernetes

What “Infrastructure as Code” means here

- **Plan first —** review what will be created before applying.
- **Same recipe —** network and cluster configuration live in versioned Terraform files.
- **Clean exit —** destroy is part of the lifecycle, not an afterthought.

```
OpsFlow on  main via 🐍 v3.12.14 (.venv) took 4s
❯ terraform -chdir=terraform apply opsflow-destroy.tfplan
module.eks.module.eks_managed_node_group["demo"].aws_iam_role.this[0]: Destroying... [id=opsflow-eks-demo-eks-node-group-20261016121314169100000002]
module.eks.aws_iam_role.this[0]: Destroying... [id=opsflow-eks-cluster-20261016121314169000000001]
module.eks.module.eks_managed_node_group["demo"].aws_iam_role.this[0]: Destruction complete after 2s
module.eks.aws_iam_role.this[0]: Destruction complete after 3s

Apply complete! Resources: 0 added, 0 changed, 2 destroyed.

OpsFlow on  main via 🐍 v3.12.14 (.venv) took 4s
❯
```

Kubernetes, Helm and GitOps

Kubernetes runs the services; Helm describes the deployment; ArgoCD keeps the cluster aligned with Git.

Kubernetes

Keeps application services running, restartable and reachable.

Helm

Packages the deployment rules so the same application can be installed consistently.

ArgoCD

Watches Git and reconciles the cluster toward the version stored in the repository.

Current GitOps flow

GitHub
main
Source of truth

ArgoCD
Synced + Healthy

Helm chart
helm/opsflow

Minikube
namespace: opsflow

Verified state

Application	Synced / Healthy	Replicas	2 backend + 2 frontend
Database	PostgreSQL PVC preserved	Ingress	opsflow.local

Video-friendly GitOps demonstration: a tiny Git change can be pushed, then ArgoCD shows the new revision synchronized into the cluster.

Observability

The system reports its own health so problems can be seen before users have to explain them.

Prometheus

Collects application metrics from both backend replicas every 15 seconds.

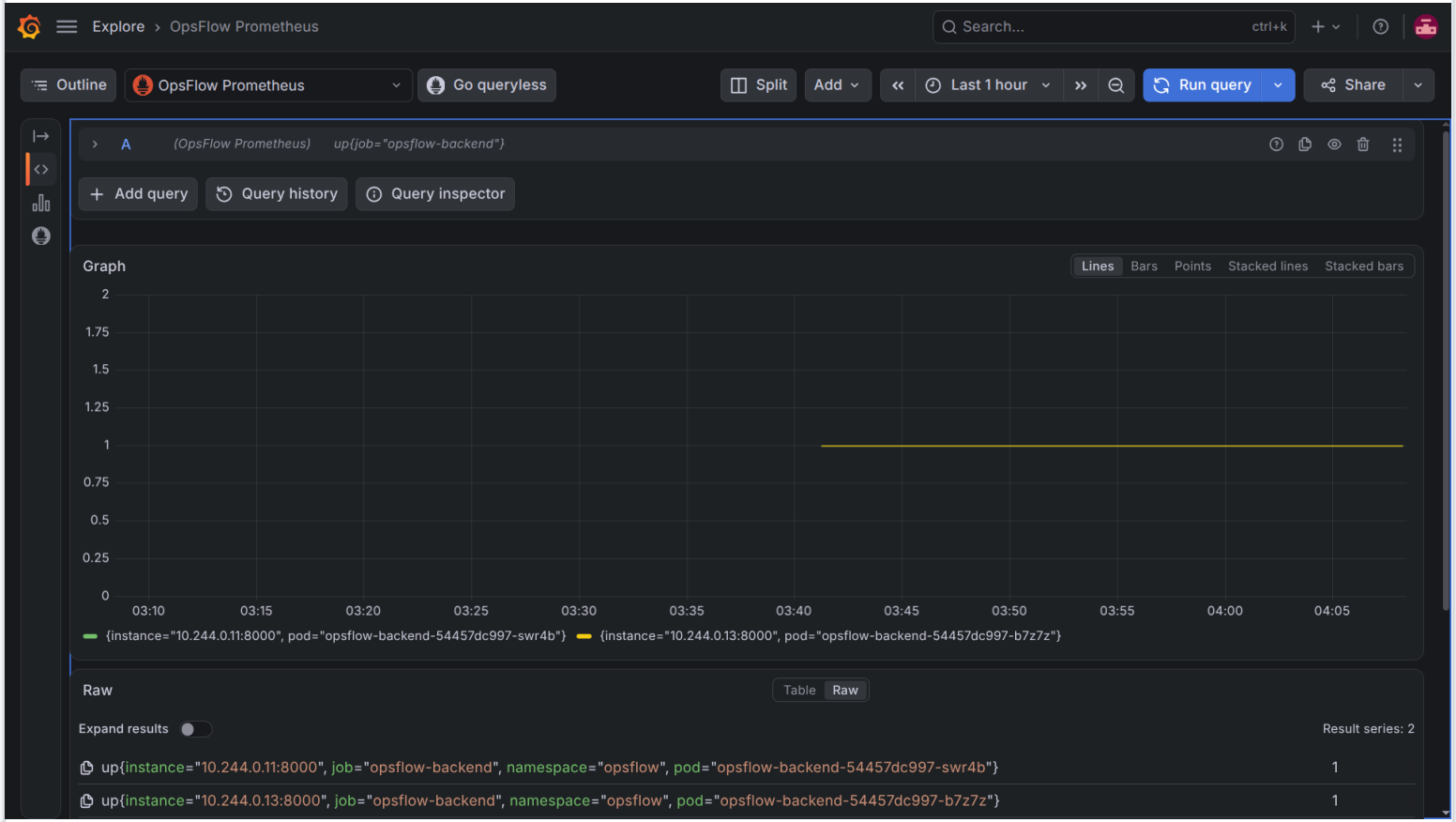
Grafana

Turns raw metrics into a dashboard for availability, traffic, latency and process behavior.

Operational endpoints

/health, /ready and /metrics provide machine-readable status checks.

2 backend targets observed as UP



Example: both backend replicas return an “up = 1” signal.

What was validated

The project was not only assembled — each layer was exercised and captured with evidence.

36

API tests passed

2 + 2

backend / frontend replicas

HIGH / CRITICAL

security gate

Synced + Healthy

ArgoCD state

AWS EKS

real cloud validation

Destroyed

cloud cleanup completed

Result: one repository now contains the application, delivery automation, infrastructure code, Kubernetes deployment, GitOps configuration, monitoring setup and grading evidence.

Key learnings

The strongest lessons are about making systems repeatable, observable and safe to change.

1 **Automate the boring checks** — Tests and security scans are more reliable when every push follows the same path.

2 **Git should explain the system** — Application, infrastructure, deployment and GitOps configuration are all reviewable as code.

3 **Health is part of the product** — Readiness checks, metrics and dashboards make operations less dependent on guesswork.

4 **Cloud resources need a lifecycle** — Provisioning is only half the job; cleanup and cost control are part of engineering quality.

5 **Keep the story understandable** — A strong system can still be explained in simple terms: build, check, deploy, synchronize, observe.

The goal is not more tools — it is a safer, clearer path from code to a running service.

Live walkthrough

The recording will now move from the slides to the real project, in the same order requested for submission.

1	**Codebase**	Show the repository structure and key files
2	**CI/CD**	Trigger a push and follow GitHub Actions end-to-end
3	**Terraform + AWS proof**	Show the saved plan and captured VPC / EKS evidence
4	**ArgoCD**	Show Synced / Healthy and the GitOps resource tree
5	**Kubernetes**	Show pods, services and ingress healthy
6	**Prometheus**	Show OpsFlow targets UP
7	**Grafana**	Show live application metrics